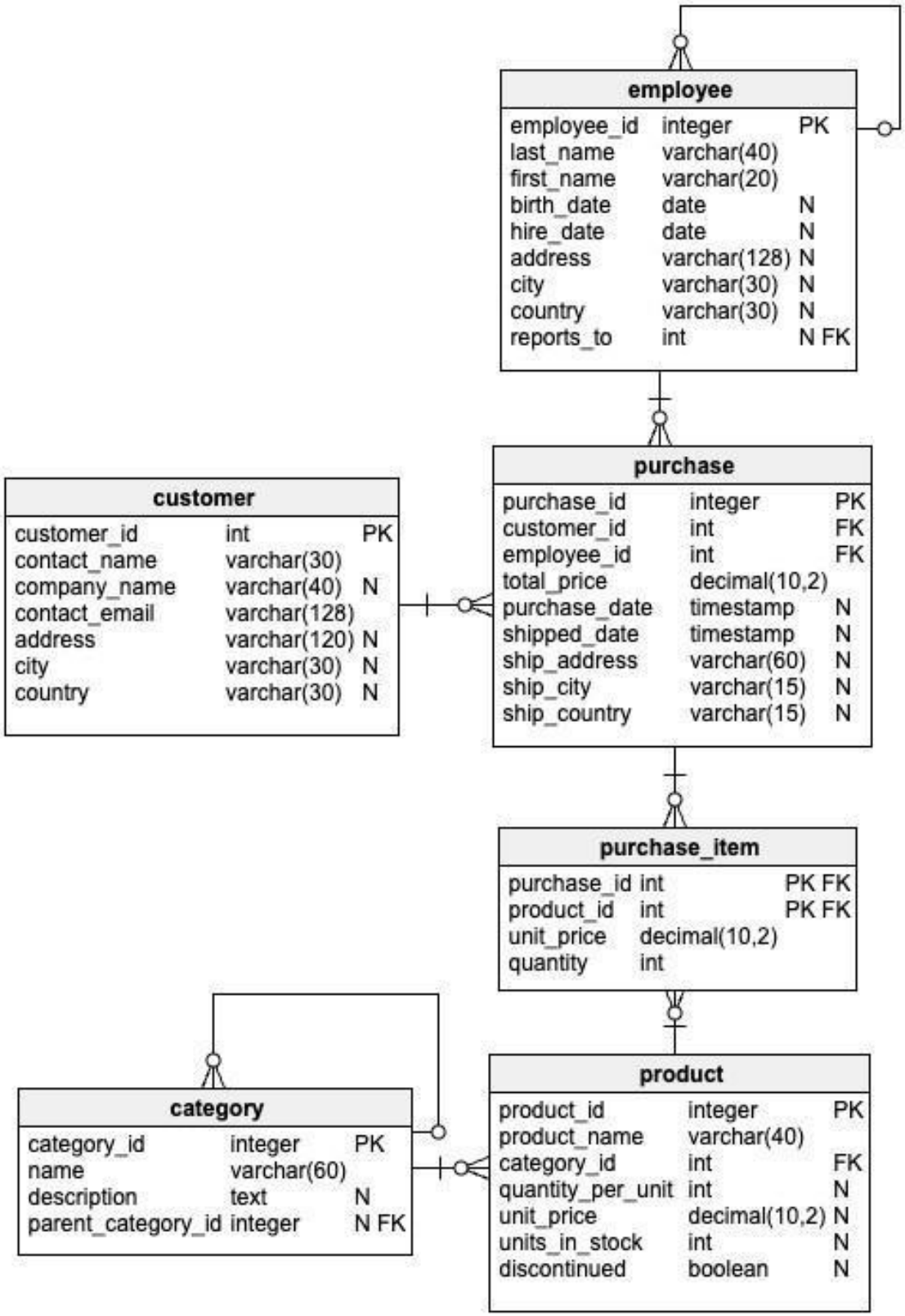

employee
employee_id integer PK
last_name varchar(40)
first_name varchar(20)
birth_date date N
hire_date date N
address varchar(128) N
city varchar(30) N
country varchar(30) N
reports_to int N FK
customer
customer_id int PK
contact_name varchar(30)
company_name varchar(40) N
contact_email varchar(128)
address varchar(120) N
city varchar(30) N
country varchar(30) N
purchase
purchase_id integer PK
customer_id int FK
employee_id int FK
total_price decimal(10,2)
purchase_date timestamp N
shipped_date timestamp N
ship_address varchar(60) N
ship_city varchar(15) N
ship_country varchar(15) N
purchase_item
purchase_id int PK FK
product_id int PK FK
unit_price decimal(10,2)
quantity int
category
category_id integer PK
name varchar(60)
description text N
parent_category_id integer N FK
product
product_id integer PK
product_name varchar(40)
category_id int FK
quantity_per_unit int N
unit_price decimal(10,2) N
units_in_stock int N
discontinued boolean N

As you can see, the store database has 6 tables:

The **customer** table contains information about the customers. It has the following columns:

- customer_id: The customer's unique internal ID.
- contact_name: The customer's full name.
- company_name: The name of the customer's company.
- contact_email: The customer's email address.
- address: The customer's street address.
- city: The city where the customer lives.
- country: The country where the customer lives.

The **product** table contains a list of products available in the store. Its columns are:

- product_id: The product's ID.
- product_name: The product's name.
- category_id: The ID of the product's category. This connects with the category
- quantity_per_unit: The quantity of product items in one unit.
- unit_price: The price of the product.
- units_in_stock: The number of the available units of the product.
- discontinued: Whether the product is available in the store (a FALSE value) or has been discontinued (TRUE).

The **category** table contains information about the categories of the products:

- category_id: The ID of the category.
- name: The category name.
- description: A description of the category.
- parent_category_id: If this category is a subcategory, this points to the parent category. Otherwise, this column will be NULL.

The **purchase** table contains information about each order:

- purchase_id: The ID of that purchase.

- customer_id: The ID of the customer who made the purchase.
- employee_id: The ID of the employee who took care of the order.
- total_price: The total price of the order.
- purchase_date: The timestamp when the order was received.
- shipped_date: The timestamp when the order was shipped.
- ship_address: The street address to which the order was shipped.
- ship_city: The city to which the order was shipped.
- ship_country: The country to which the order was shipped.

The **purchase_item** table connects purchases with products. It has the following mandatory columns:

- purchase_id: The purchase ID.
- product_id: The ID of the purchased product.
- unit_price: The price of one unit of that product.
- quantity: The number of purchased units of that product.

The **employee** table stores information about store employees. It has the following columns:

- employee_id: The ID of the employee.
- last_name: The employee’s last name.
- first_name: The employee’s first name.
- birth_date: The employee’s date of birth.
- address: The employee's street address.
- city: The employee's city.
- country: The employee's country.
- reports_to: The ID of that employee’s direct supervisor. It's NULL if the employee doesn't report to anyone.

1. All Products

Exercise

Display all data in the product table.

Solution

```
SELECT *
FROM product;
```

Explanation

In this query, the SELECT clause is used to specify the columns that we want to retrieve from the table product. In this case, the asterisk (*) is shorthand for “all columns”.
The FROM clause specifies the table from which the data is to be retrieved. In this query, it's the product table.

Putting it all together, the query is essentially asking the database to return all columns for every row in the **product** table.

2. Names and Emails of Customers

Exercise

Display the names and email addresses of the customers. You'll find the name in the contact_name column and the email in the contact_email column. Rename the columns to name and email.

Solution

```
SELECT
 contact_name AS name,
 contact_email AS email
FROM customer;
```

Explanation

In this query, the SELECT clause is used to specify the columns that we want to retrieve from the table customer. Note that the columns are separated by commas.

Then, we use the AS keyword to rename the column contact_name to name and contact_email to email. These new names are called “aliases”.

As previously, the FROM clause is used to indicate the table from which the data is going to be retrieved; here, customer.

Putting it all together, the query asks the database to retrieve the contact_name and contact_email columns from the customer table. It also asks it to display these columns as name and email, respectively. The result set will consist of these two aliased columns for each row in the customer table.

3. Customers with a Purchase

Exercise

Display the IDs of all customers who made at least one purchase. No duplicated customer IDs should be shown.

Solution

```
SELECT DISTINCT customer_id

FROM purchase;
```

Explanation

Since we want to retrieve customers who've made a purchase, we select their IDs from the table that stores purchase data, purchase. The SELECT clause combined with DISTINCT removes any duplicate customer_id values from the result set.

4. Customer with ID 4

Exercise

Display all data for a customer with ID 4.

Solution

```
SELECT *

FROM customer

WHERE customer_id = 4;
```

Explanation

This query retrieves all columns (* represents all columns) from the customer table – but only where the value in the customer_id column is equal to 4.

The WHERE clause filters the rows to include only those that match the stated condition – here, that the value in the customer_id column is equal to 4. The = operator is used for equality comparison. The result set will include all columns of the rows that meet this condition.

5. Customers from Dallas

Exercise

Display the names of all customers living in Dallas.

Solution

```
SELECT contact_name

FROM customer

WHERE city = 'Dallas';
```

Explanation

This query retrieves all columns (*) from the customer table. The WHERE clause filters the rows to include only those where the value in the city column is equal to 'Dallas'. 'Dallas' is a string value, so it's enclosed in single quotes.

6. Products Starting with "Beef"

Exercise

Display all data for products whose names start with 'Beef'.

Solution

```
SELECT *

FROM product

WHERE product_name LIKE 'Beef%';
```

Explanation

The WHERE condition filters the rows to include only those where the value in the product_name column starts with 'Beef'. The LIKE operator is used for pattern matching: 'Beef%' is a pattern where Beef is the start of the pattern and % is a wildcard that matches any sequence of characters following 'Beef'.

Note that the LIKE operator is case sensitive; values that start with ‘beef’ or ‘BEEF’ will not be included in the results.

7. Non-Food Products

Exercise

Display product names and the related category IDs for categories with *any ID other than 1* (Food).

Solution

```
SELECT
 product_name,
 category_id
FROM product
WHERE category_id <> 1;
```

Explanation

This query retrieves specific columns (product_name and category_id) from the product table where the value in the category_id column is different from 1. The <> operator is the opposite of the = operator; it returns only values that **do not** match the condition.

8. Customers from Oloo or Fliptune

Exercise

Display the names of all customers whose company name is Oloo or Fliptune.

Solution

```
SELECT
 contact_name
FROM customer
WHERE company_name = 'Oloo'
 OR company_name = 'Fliptune';
```

Explanation

This query retrieves the customers' names from the customer table where the value in the company_name column is equal to 'Oloo' or 'Fliptune'. The OR operator is used to combine these conditions, indicating that **one** of them must be true for a row to be included in the result set.

9. Expensive Food and Fruit & Vegetable Products

Exercise

Display data for all products in the category with the ID of 1 (Food) or 5 (Fruits and vegetables) and with a unit price above 3.5.

Solution

```
SELECT *

FROM product

WHERE (category_id = 1 OR category_id = 5)

 AND unit_price > 3.5;
```

Explanation

This query retrieves all columns from the product table where the value in the category_id column is equal to 1 or 5, **and** the value in the unit_price column is greater than 3.5. The logical operators OR and AND are used to combine these conditions. Note that the OR conditions are surrounded by parentheses to tell SQL to execute this block first.

If you want to practice SQL using hands-on exercises, try our 100% online SQL Practice track.

10. Active Products Not in Category 3

Exercise

Display the name of all products except for non-discontinued products belonging to category ID 3.

Solution

```
SELECT product_name

FROM product
```

```
WHERE NOT (discontinued IS TRUE AND category_id = 3);
```

Explanation

This query retrieves all columns from the product table *except* those where the value in the category_id column is equal to 3, and the logical value in the discontinued column is TRUE. The NOT keyword negates the condition surrounded by parentheses, meaning it selects rows where discontinued and category_id is anything other than TRUE and 3.

11. Purchases by Shipping Date

Exercise

Display all data for purchases ordered by shipping date. Purchases with more recent shipping dates must be displayed first.

Solution

```
SELECT
 purchase_id,
 total_price,
 shipped_date
FROM purchase
ORDER BY shipped_date DESC;
```

Explanation

This query retrieves the purchase_id, total_price, and purchase_date columns from the purchase table and orders the result set based on the shipped_date column in descending order. The ORDER BY clause orders the result set based on the values in the shipped_date column. The DESC keyword stands for "descending", which means the result set will be sorted in descending order (from the most recent dates to the earliest dates).

12. High-Value Purchases

Exercise

Display all data for purchases with a total price greater than or equal to 10. Purchases with the most recent shipping dates must be displayed first.

Solution

```
SELECT *
FROM purchase
WHERE total_price >= 10
ORDER BY shipped_date DESC;
```

Explanation

This query retrieves all columns from the purchase table where the value in the total_price column is greater than or equal to 10. It also orders the result set based on the shipped_date column in descending order (DESC). The WHERE condition filters the rows to include only those where the value in the total_price column is greater than or equal to 10.

13. Count of Products in Categories

Exercise

Display all category IDs and the number of products in that category.

Solution

```
SELECT category_id, COUNT(*)
FROM product
GROUP BY category_id;
```

Explanation

This query retrieves the count of products in each category in the product table.
The SELECT clause specifies that we want to retrieve the category_id column and the count of occurrences of each distinct category_id (the aggregate function COUNT(*)).

Finally, the GROUP BY clause groups the results based on the values in the category_id column. It ensures that the count is calculated for each unique category_id.

14. Average Purchase Price by Customer ID

Exercise

For all customers, display the customer ID and the average price of all purchases for that customer.

Solution

```
SELECT
 customer_id,
 AVG(total_price)
FROM purchase
GROUP BY customer_id;
```

Explanation

This query is similar to the previous exercise – but this time, we use the aggregate function AVG() to display the average of the purchases for each customer.
The SELECT clause returns the customer_id column and the average of the total_price for each distinct customer_id in the purchase table.

15. Total Purchases by Customer–Employee Pair

Exercise

For each customer–employee pair, find the total price of purchases made by this customer and handled by this employee. Display three columns: customer_id, employee_id, and the total_price of purchases. Name the third column total_purchases_price.

Solution

```
SELECT
 customer_id,
 employee_id,
 SUM(total_price) AS total_purchases_price
FROM purchase
GROUP BY
 customer_id,
 employee_id;
```

Explanation

This query retrieves the total prices of purchases for each customer–employee pair.
The SELECT clause includes here the customer_id column, the employee_id column, and the sum of the total_price for each customer_id in the purchase table. The GROUP

BY clause groups the results based on the values in
the customer_id and employee_id columns.

16. Minimum Purchase Amount for Customer–Employee Pairs

Exercise

Find the smallest purchase amount for each customer–employee pair. Show only pairs for which there are at least two purchases. The columns' names should
be customer_id, employee_id, and minimum_price.

Solution

```
SELECT
 customer_id,
 employee_id,
 MIN(total_price) AS minimum_price
FROM purchase
GROUP BY customer_id,
 employee_id
HAVING COUNT(total_price) >= 2;
```

Explanation

This query retrieves the distinct values in the customer_id and employee_id columns along with the minimum purchase price located in the column total_price. The minimum purchase price is named minimum_price. The GROUP BY clause groups the results based on the values in the customer_id and employee_id columns, and the HAVING clause filters the results to include only those groups where the count of occurrences is greater than or equal to 2.

17. Employee Names with Purchase Shipment Dates

Exercise

Show the last and first names of the employees taking care of each purchase and
the shipped_date of that purchase.

Solution

```
SELECT
 last_name,
 first_name,
 shipped_date
FROM employee
JOIN purchase
 ON employee.employee_id = purchase.employee_id;
```

Explanation

This query performs an INNER JOIN operation between the employee and purchase tables and selects specific columns (last_name, first_name, and shipped_date) from the joined tables. It combines rows from the employee table with matching rows from the purchase table based on the common column employee_id. If there is no match in the purchase table, the row won't be included in the result set.

18. Customer Details with Purchase History

Exercise

Display the names and emails of all customers and list the names of the products they bought and the product price at the time of purchase. Include customers who haven't made any purchases. Don't rename the columns.

Solution

```
SELECT
 contact_name,
 contact_email,
 product_name,
 purchase_item.unit_price
FROM customer
LEFT JOIN purchase
 ON customer.customer_id = purchase.customer_id
```

```
LEFT JOIN purchase_item
  ON purchase.purchase_id = purchase_item.purchase_id
LEFT JOIN product
  ON purchase_item.product_id = product.product_id;
```

Explanation

This query performs a triple LEFT JOIN operation between the customer**,** purchase**,** purchase_item, and product tables and selects specific columns (contact_name, contact_email, product_name, and unit_price) from the result set. It combines rows from the customer table with matching rows from the purchase**,** purchase_item, and product tables based on the common columns customer_id, purchase_id, and product_id.

If there is no match in the purchase table, NULL values will be included in the result set for the columns from the purchase_item table.

19. Categories with Discontinued Products

Exercise

For each category, find the number of discontinued products. Show only the categories with at least three discontinued products. Sort the rows by the number of discontinued products in descending order. Display two columns: name (the name of the category) and discontinued_products_number.

Solution

```
SELECT
  category.name,
  COUNT(product_id) AS discontinued_products_number
FROM product
JOIN category
  ON product.category_id = category.category_id
WHERE discontinued IS TRUE
GROUP BY category.name
```

```
HAVING COUNT(product_id) >= 3
ORDER BY COUNT(product_id) DESC;
```

Explanation

This query performs an INNER JOIN operation between the product and category tables. It displays the category name and the count of occurrences of each distinct product_id in the result set. It combines rows from the product table with matching rows from the category table based on the common column category_id.

Rows are grouped by category names; the aggregate function COUNT() is used on the category_id column to return the number of products in that category. The HAVING clause filters for categories with at least discontinued products. Rows are displayed in descending order of product ID (ORDER BY COUNT(product_id) DESC).

20. Customers with High-Value Purchases

Exercise

For each customer, show the number of purchases they've made. Include only purchases with a non-NULL ship_city and show only customers whose total cost for all purchases was over 14. The columns' names should be contact_name and purchase_quantity. Sort the rows by contact_name.

Solution

```
SELECT
 contact_name,
 COUNT(*) AS purchase_quantity
FROM purchase
JOIN customer
 ON purchase.customer_id = customer.customer_id
WHERE ship_city IS NOT NULL
GROUP BY contact_name
HAVING SUM(total_price) > 14
```

```
ORDER BY contact_name;
```

Explanation

This query performs an INNER JOIN operation between the purchase and customer tables and selects the contact_name and the count of occurrences of each distinct purchase from the result set. Rows are grouped by contact names. The HAVING clause allows filtering for the required total price.